GLB 3rd Year Assessment 2 - Slot 1

Contest Problem Report

Contest Code	GLB3A2S1
Total Problems	5
Coding Problems	5

Problem Name: Count of set bits

Problem Statement:

Checking set bit

To check if a specific bit is set (equals 1) or not in a binary representation, you can use the bitwise AND operator (&). Here's how you can do it in pseudocode:

```
function is_bit_set(number, position):
# Shift 1 to the left by the specified position
mask = 1 << position
# Use bitwise AND to check if the bit is set
return (number & mask) != 0
```

In this pseudocode:

- 1 << position creates a mask with a 1 at the specified position by left-shifting the binary digit 1 to the left by the given position.
- (number & mask) != 0 checks if the bit at the specified position is set by performing a bitwise AND operation. If the result is not zero, it means the bit is set.

Task

Write a program that takes an input integer N and prints the count of the set bits present in the number.

Input Format

- The first line of input will contain a single integer *N*, denoting the number.

Output Format

Output on a single line, the number of set bits in *N*.

Constraints

- $1 \leq N \leq 1000000000$

Sample Input 1:

256

Sample Output 1:

1

Explanation:

Binary representation of 256 is: 100000000 Number of set bits = 1

Sample Input 2:

15

Sample Output 2:

4

Explanation:

Binary representation of 15 is: 1111 Number of set bits = 4

Problem Name: Longest Palindromic Substring

Problem Statement:

Chef has a string **S** consisting of letters and digits.\ He wants to find the **longest palindromic substring**.

A **palindromic substring** is a contiguous sequence of characters that reads the same forward and backward. Due to case sensitivity 'A' != 'a' in a palindrome.

The function searches for all possible palindromic substrings in *s* and returns the one with the maximum length.
If there is a tie, priority is given to the substring with the **smallest starting index**.

Function Declaration

Function Name

findLongestPalindrome – This function finds the longest palindromic substring in a given string.

Parameters

- *s* : A string consisting of English letters (uppercase and lowercase) and digits.

Return Value

- Returns a string representing the **longest palindromic substring** in *s*.
- If multiple palindromic substrings of the same maximum length exist, the one that appears **first** in *s* is returned.

Constraints

- $1 \leq |s| \leq 1000$
- String *s* consists of:
    - English letters (uppercase and lowercase)
    - Digits (0–9)

Input Format

- A single line containing the string *s*.

Output Format

- Print the longest palindromic substring.
- If multiple answers exist, print the one that appears first in the string.

Sample Input 1:

racecar12321

Sample Output 1:

racecar

Explanation:

"racecar" and "12321" are both palindromes.

Both have length 7 and 5 respectively.

"racecar" length is larger so it is printed.

Sample Input 2:

abaxyzzyxf

Sample Output 2:

xyzzyx

Explanation:

The longest palindromic substring is "xyzzyx" (length 6).

Even though "aba" is a palindrome, it is shorter than "xyzzyx".

Sample Input 3:

121323

Sample Output 3:

121

Explanation:

121 and 323 are both palindrome.

Both have length of 3.

Thus for tie breaker the palindrome with the smaller starting index will be printed.

121 starting index=0. 323 starting index=3.

Problem Name: Minimum number of coins

Problem Statement:

Chef has infinite coins in denominations of rupees *5* and rupees *10*.

Find the **minimum** number of coins Chef needs, to pay **exactly** *X* rupees. If it is impossible to pay *X* rupees in denominations of rupees *5* and *10* only, print *–1*.

Input Format

- First line will contain *T*, number of test cases. Then the test cases follow.
- Each test case contains of a single integer *X*.

Output Format

For each test case, print a single integer - the **minimum** number of coins Chef needs, to pay **exactly** *X* rupees. If it is impossible to pay *X* rupees in denominations of rupees *5* and *10* only, print *–1*.

Constraints

- $1 \leq T \leq 1000$
- $1 \leq X \leq 1000$

Subtasks

- **Subtask 1 (100 points):** Original constraints.

Sample Input 1:

3 50 15 8

Sample Output 1:

5 2 -1

Explanation:

Test Case 1: Chef would require at least *5* coins to pay *50* rupees. All these coins would be of rupees *10*.

Test Case 2: Chef would require at least *2* coins to pay *15* rupees. Out of these, *1* coin would be of rupees *10* and *1* coin would be of rupees *5*.

Test Case 3: Chef cannot pay exactly *8* rupees in denominations of rupees *5* and *10* only.

Problem Name: Delete All Occurrences of a Key in a Doubly Linked

Problem Statement:

You are given a **doubly linked list** consisting of *N* nodes and an integer value *X*.\ Your task is to **delete all nodes** from the linked list whose value is equal to *X*, and then **return the modified linked list**.\ If after deletion the linked list becomes empty, print *-1*.

Function Declaration

Function Name

deleteAllOccurrences – This function deletes all nodes with a given value from a doubly linked list.

Parameters

- *head* : Pointer to the head of the doubly linked list.
- *X* : An integer value; all nodes with this value must be deleted.

Return Value

- Returns the **head of the modified doubly linked list**.
- Returns *NULL* if all nodes are deleted (caller prints *-1*).

The input and output formats given below are only if you want to test using custom inputs.

Constraints:

- $0 \leq N \leq 10^5$
- $-10^4 \leq __TEXT_PLACEHOLDER0 \leq 10^4$
- $-10^4 \leq X \leq 10^4$

Input Format

- The first line contains an integer *N* — the number of nodes in the linked list.
- The second line contains *N* space-separated integers — the elements of the linked list.
- The third line contains an integer *X* — the value to be deleted from the list.

Output Format

- Print the elements of the modified linked list after deleting all occurrences of *X*.
- If the list becomes empty, print *-1*.

Sample Input 1:

6 10 20 30 20 40 20 20

Sample Output 1:

10 30 40

Explanation:

All nodes with the value `20` are deleted.\ The new list becomes: `10 <-> 30 <-> 40`.

Sample Input 2:

5 5 15 25 35 45 50

Sample Output 2:

5 15 25 35 45

Explanation:

The value `50` is not present, so the list remains unchanged.

Problem Name: Music Festival Playlist

Problem Statement:

A college music festival has a playlist that the DJ controls during the event.

The playlist is circular:

- Every song has a unique integer ID.
- The DJ can move to the next or previous song.
- Moving forward from the last song takes the DJ to the first song.
- Moving backward from the first song takes the DJ to the last song.

The initial songs are given in clockwise order. The DJ starts with the first song selected.

You are given a sequence of operations that modify or navigate the playlist:

ADD x: Add a song with ID x immediately after the selected song in clockwise order. The new song becomes selected. If the playlist is empty, create a one-song playlist containing x and select it.

REMOVE: Remove the selected song. The next song in clockwise order becomes selected. If the removed song was the only song, the playlist becomes empty.

NEXT: Select the next song in clockwise order.

PREV: Select the previous song.

REMOVE, NEXT, and PREV are issued only when the playlist is nonempty. Moving to the next or previous song in a one-song playlist keeps that song selected. Only ADD can be issued when the playlist is empty.

After all operations, print EMPTY if the playlist is empty. Otherwise, print every song ID once, starting with the selected song and continuing clockwise. Do not print the selected song a second time.

Input Format

The first line contains an integer n, the number of songs initially in the playlist.

The second line contains n space-separated integers, the initial song IDs in clockwise order.

The third line contains an integer q, the number of operations.

Each of the next q lines contains one operation: ADD x, REMOVE, NEXT, or PREV.

All song IDs are unique while they are present in the playlist. An ID may be added again after its song has been removed.

Output Format

If the final playlist is empty, print EMPTY.

Otherwise, print one line of space-separated song IDs in clockwise order, starting from the selected song. Print every song exactly once.

Constraints

- 1 ≤ n ≤ 1000
- 1 ≤ q ≤ 100000
- 1 ≤ song ID ≤ 1000000000
- Initial song IDs are distinct.
- For ADD x, x is not already present in the playlist.
- All operations are valid under the rules above.

Sample Input 1:

4 10 20 30 40 3 NEXT ADD 50 NEXT

Sample Output 1:

30 40 10 20 50

Explanation:

NEXT selects 20. ADD 50 inserts 50 after 20 and selects it. The final NEXT selects 30.

Starting from the selected song 30 and moving clockwise, the playlist is 30 40 10 20 50.

Sample Input 2:

3 1 2 3 2 NEXT REMOVE

Sample Output 2:

3 1

Explanation:

Initially, 1 is selected. NEXT selects 2. REMOVE deletes 2, and the next song clockwise, 3, becomes selected.

Starting from 3, the remaining playlist is 3 1.

Sample Input 3:

1 100 1 REMOVE

Sample Output 3:

EMPTY

Explanation:

The playlist contains only one song. Removing the selected song makes the playlist empty, so the output is EMPTY.